Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	13 February 2026
Team ID	
Project Name	Malaria Detection using Deep Learning
Maximum marks	4

1. Technical Architecture

The system is designed using a **web-based AI-driven architecture** where users upload blood smear images, and the trained deep learning model processes and predicts malaria infection.

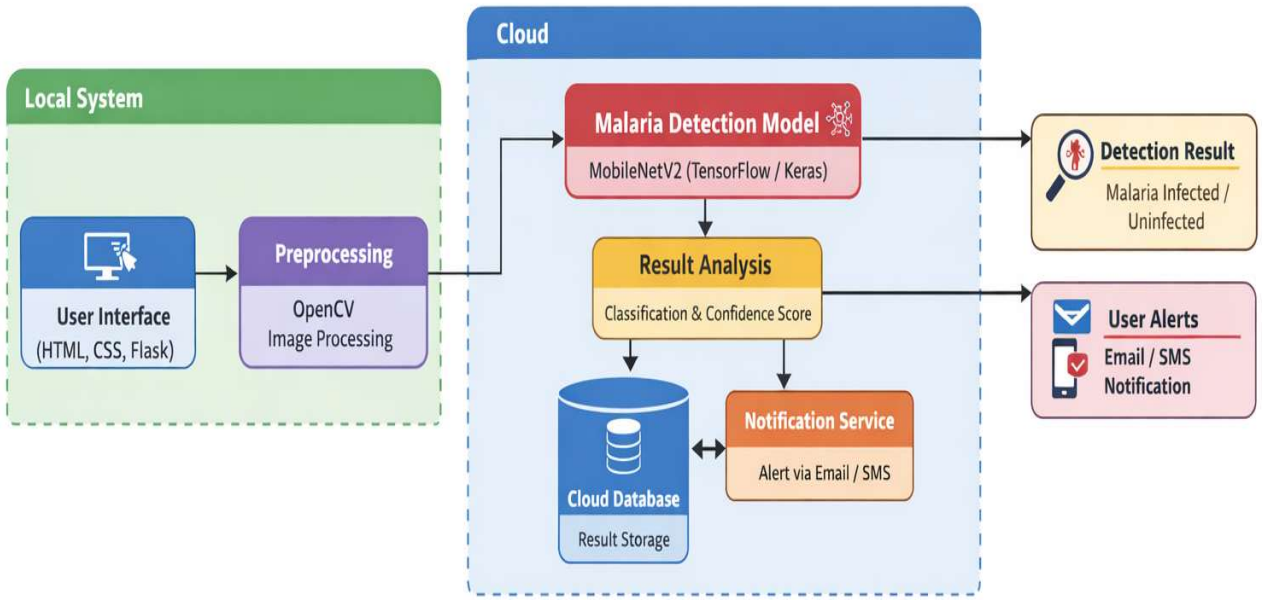

Figure 1.1 Architecture

Architecture Overview:

1. User uploads blood smear image via web interface
2. Image preprocessing and normalization
3. Deep learning model (MobileNetV2) processes image
4. Prediction result generated
5. Result displayed on web interface

This architecture enables automated, fast, and accurate malaria detection.

S.No	Component	Description	Technology
1	User Interface	Web-based interface for image upload	HTML, CSS, Flask
2	Application Logic	Image processing and prediction	Python
3	Machine Learning Model	Deep learning for malaria detection	MobileNetV2 (TensorFlow/Keras)
4	Database (if required)	Store user history (optional)	SQLite / MySQL
5	File Storage	Store uploaded images	Local filesystem
6	Infrastructure	Deployment environment	Local server
7	External API	Not applicable (self-contained system)	Not used
8	Cloud Deployment (optional)	Online hosting	Render / AWS / Heroku

Table 1.1 Components and Technology

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Frameworks used for development	TensorFlow, Flask, OpenCV
2	Security Implementations	Basic security for uploads	File validation, input sanitization
3	Scalable Architecture	Supports expansion for larger datasets	Modular design
4	Availability	Web-based access	Local server / Cloud deployment
5	Performance	Optimized image processing	MobileNetV2 (efficient CNN)

Table 1.2 Application Characteristics

2. Infrastructure Demarcation

- Local Environment: Model training and testing
- Web Deployment (Optional): Flask-based server
- Storage: Local filesystem for uploaded images
- Processing: Deep learning model execution on server

3. External Interfaces

- Web UI for image upload
- Deep learning model for prediction
- No third-party APIs required

4. Data Storage Components

- Uploaded images stored in local directory
- Model file (.h5) for predictions
- Optional database for history tracking

5. Machine Learning Interface

- TensorFlow/Keras model integration
- Image preprocessing before prediction
- Output classification (Parasitized / Uninfected)